ReliableML: A Leakage-Aware Library for Evidence of Classifier Reliability

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

Olorato Motitswane
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
oloratomotitswane@gmail.com

Mfundo Monchwe
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
diditmfundo@gmail.com

***Abstract*—ReliableML is an open-source (MIT) Python library that treats the reliability of a classifier as a body of evidence rather than a single held-out score. It provides statistical data cleaning, ATTP-based partitioning through an optional dependency, cross-fitted meta-learning classifiers, a pure-NumPy model family (StatCore), labeled evaluation with uncertainty, reference-based drift monitoring, fairness estimands, temporal deterioration assessment, explanation-consistency checks, a statistical reason audit, and a `ReliablePipeline` that couples a fitted model with a fit-time reference so that predictions can be accompanied by reports. An optional companion package, ReliableML-AutoML, automates candidate selection on top of these parts. This paper describes the architecture, the recommended order of operations, the design principles, and the scope and limitations. It reports no benchmark results; method-specific derivations are in the per-module papers and a comparative AutoML analysis is in a separate paper.**

***Index Terms*—machine learning reliability, data leakage, model evaluation, distribution shift, tabular classification**

I. Motivation

A single accuracy figure on one held-out sample says little about whether a deployed classifier can be trusted. Evidence is needed about the data (were they cleaned without leaking validation rows?), the model (how was it selected, and how optimistic is that selection?), the operating environment (has the input or the input–label relation shifted?), and the consequences (are errors distributed fairly, and do explanations agree with the model's behaviour?). ReliableML packages these checks behind a common estimator-style interface so that they can be composed into one workflow, and so that each step states what it does and does not establish.

II. Architecture

The library is organised as modules that can be used independently. Table I lists them with their principal purpose.

A. Cleaning

The cleaner decides imputation per column from the observed distribution: the mode for columns with at most two distinct values; the mean when approximate-normality criteria hold; otherwise the median; categorical columns use the mode. Outliers are located with Tukey fences and are either only flagged (light) or capped (moderate, heavy). A report records the statistics, counts, and decisions so that cleaning is auditable. Because these statistics are estimated from data, they must be fitted on training rows only; the module documents this.

B. Partitioning

ATTP partitioning is an optional dependency. It fits transformations and clustering before proposing a split and selects among candidates using its own internal test score. The resulting partition is therefore a design proposal whose internal test rows are not independent evidence; a final test set should be reserved before it is called.

C. Meta-learning

Meta-learners combine base classifiers using out-of-fold predictions so that the second-stage model is trained on predictions that did not see their own labels [1], [2]. `PerfMetaSelect` chooses among base models by cross-validated performance. Calibrated and statistical meta-classifiers add probability calibration or statistical combination

TABLE I
RELIABLEML MODULES AND THEIR ROLE.

Module	Role
Cleaning	Statistical cleaning: missing-value rules by distribution shape, IQR outlier handling at light/moderate/heavy intensity, with a record of what was changed.
ATTP partitioning (optional)	Cluster- and class-aware train/test partition proposals from the separate `attp` package; not a final evaluation.
Meta-learning	Cross-fitted stacking and performance-aware selection (`MetaPipelineClassifier`, `PerfMetaSelect`, calibrated stacks, statistical meta-classifiers).
StatCore	Pure-NumPy PCA, LDA, K-means, logistic/ridge, KNN, MLP and meta-learner with a shared `variant` and `evaluate` interface, plus sklearn-compatible adapters.
Evaluation	Labeled metrics with interval estimates and statistical tests.
Explainability	Global and local explanation consistency.
Deterioration	Trend assessment of performance over time.
Drift	Reference-based covariate and concept-shift monitoring.
Fairness	Group-wise fairness estimands.
Reason audit	Statistical audit of which factors accompany errors.
Pipeline	`ReliablePipeline`: model plus fit-time reference, prediction reports, save/load.
Benchmark	Multi-dataset method comparison and summaries.
AutoML (companion)	`ReliableAutoML`: nested candidate selection over the parts above.

rules. The cost is that each meta-learner internally cross-fits its bases.

D. StatCore

StatCore is a family of models implemented only with NumPy, with a documented specification and deterministic `random_state`. The design rule is that new library code for these models does not import scikit-learn, which keeps the algorithms inspectable and the dependency surface small. sklearn-compatible adapters exist so that StatCore models can be used in scikit-learn pipelines and in the AutoML candidate pool; the adapters, not the models, depend on that interface.

E. Evaluation, drift, deterioration, fairness

Labeled evaluation reports classification metrics with uncertainty and tests. Drift monitoring compares new features to a fit-time reference profile (and, when labels are available, assesses concept shift by feature). Deterioration assessment tests for a trend in a performance history. Fairness reporting computes group-wise estimands on user-supplied sensitive attributes. Each of these produces a report object rather than a pass/fail flag; none certifies safety or fairness.

F. Explainability and reason audit

Explanation checks measure agreement between global and local explanations. The reason audit relates recorded reasons or influential features to correctness of predictions, to surface factors that accompany errors. Both are statistical summaries of model behaviour, not causal explanations.

G. ReliablePipeline

`ReliablePipeline.fit` fits a drift monitor on the reference features and then the estimator. `predict_with_report` returns predictions together with an unlabeled drift report; the pipeline also exposes `evaluate`, shift and deterioration assessment, explanations and reason audits. `save` serialises the pipeline with pickle. The artefact may retain reference observations and must be treated as sensitive and loaded only from trusted sources.

III. RECOMMENDED WORKFLOW

W1. Reserve an untouched final test set (grouped, temporal or site-aware if the data demand it) before any decision is informed by labels.

W2. On development data only, validate schema and clean (fit-statistics from training rows only).

W3. Optionally obtain a training/validation partition (ATTP) from development data.

W4. Fit and select candidates, including meta-learners and StatCore models, with nested or otherwise fold-local selection.

W5. Fit `ReliablePipeline` on the training partition to retain the fit-time reference.

W6. Evaluate once on the final test set; then report fairness, explanations, and reason audit.

W7. In operation, call `predict_with_report`, monitor drift and deterioration, and revisit the model when evidence warrants.

`ReliableAutoML` automates steps W2–W5 and does not replace W1 or W6.

IV. DESIGN PRINCIPLES

Leakage control: statistics that depend on data are fitted inside folds; where a documented exception exists (the partition-design encoding in AutoML) it is stated. *Evidence, not verdicts:* modules return reports with uncertainty and warnings. *Auditability:* cleaning and selection decisions are recorded. *Small dependency surface:* StatCore uses NumPy only; ATTP and AutoML are optional. *Honest scope:* selection among candidates remains optimistic even when nested [3], [4]; only fresh test data remove that optimism.

V. SCOPE AND LIMITATIONS

The library targets tabular classification. It does not guarantee calibration, robustness, fairness or safety, and its reports are statistical diagnostics whose power depends on sample size. Cleaning and ATTP assumptions (finite numeric features for S-ATTP, approximate-normality heuristics) can be inappropriate for some data. Pickle artefacts are unsafe to load from

untrusted sources. The StatCore models are for transparency and control and are not claimed to outperform optimised third-party implementations; the AutoML companion's measured behaviour against other frameworks is the subject of the comparison paper. Datetime columns and temporal dependence need user-designed splits.

REFERENCES

[1] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[2] L. Breiman, "Stacked regressions," *Machine Learning*, vol. 24, pp. 49–64, 1996.

[3] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, article 91, 2006.

[4] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *JMLR*, vol. 11, pp. 2079–2107, 2010.

[5] J. Monchwe *et al.*, ReliableML, software repository, 2026. https://github.com/Jabualani/ML-Reliabilty.